

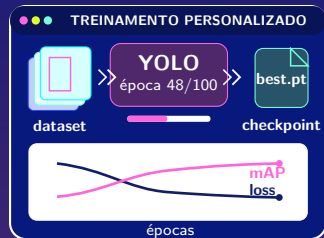
Aula 09

Treinando nosso próprio YOLO

Dos dados anotados ao detector personalizado

Treinar, validar e investigar

Francisco Leonel Ferreira dos Santos
Treinamento de Visão Computacional com YOLO
Caxias – MA, 2026





Imagens
arquivos abrem
corretamente

Labels
caixas e clas-
ses conferidas

Divisão
train e val sem
vazamento

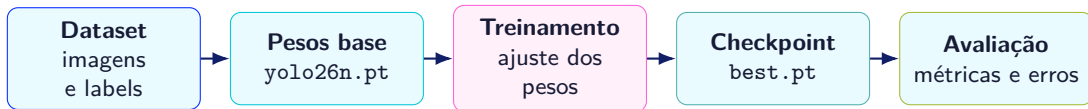
YAML
caminhos e no-
mes válidos

Ambiente
Ultralytics importada

Recursos
espaço, tempo e device

Regra prática

Execute primeiro um treinamento curto. Um erro descoberto em poucas épocas custa menos que um erro encontrado ao final.



O ciclo não termina na primeira execução

A avaliação produz hipóteses; as hipóteses orientam correções nos dados, na configuração ou no objetivo do projeto.



Treinamento do zero

Início: pesos aleatórios.

- ▶ precisa aprender características básicas;
- ▶ normalmente exige mais dados e recursos;
- ▶ pode ser útil em domínios muito diferentes.

```
YOLO("yolo26n.yaml")
```

Fine-tuning / transfer learning

Início: pesos pré-treinados.

- ▶ reutiliza características visuais;
- ▶ costuma convergir mais rapidamente;
- ▶ é a escolha prática deste treinamento.

```
YOLO("yolo26n.pt")
```

O que o modelo pré-treinado já conhece?



Características básicas

Bordas, texturas, contrastes e padrões visuais elementares.

Representações

Formas e combinações úteis para descrever objetos.

Saída adaptada

A tarefa passa a utilizar as classes definidas no novo dataset.

Ideia central

O modelo não começa sem experiência: ele adapta representações visuais aprendidas anteriormente ao novo problema.

O novo dataset pode possuir outras classes.



```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")

results = model.train(
    data="data.yaml",
    epochs=50,
    imgsz=640
)
```

1. Carregar

Inicia com pesos pré-treinados.

2. Configurar

Informa dataset, épocas e resolução.

3. Treinar

Salva métricas, gráficos e checkpoints.

Quatro argumentos para o primeiro experimento



data

Caminho do arquivo YAML que descreve o dataset e suas classes.

epochs

Quantidade máxima de passagens pelo conjunto de treinamento.

imgsz

Resolução usada para preparar as imagens de entrada.

device

Dispositivo computacional: CPU, GPU ou outro acelerador compatível.

Comece com uma configuração simples e registre cada alteração posterior.



```
yolo detect train \  
  model=yolo26n.pt \  
  data=data.yaml \  
  epochs=50 \  
  imgsz=640
```

Python

Adequado para integrar treinamento, análise e automação no mesmo programa.

CLI

Conveniente para executar e registrar rapidamente uma configuração.

Epochs: quantas vezes percorrer o conjunto de treino?



Poucas épocas

O modelo pode interromper o aprendizado antes de capturar os padrões necessários.

Faixa adequada

Treino e validação evoluem até um desempenho estável e útil para o projeto.

Épocas demais

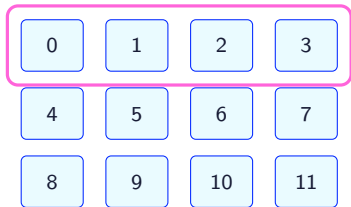
Aumentam custo e podem intensificar overfitting sem melhorar generalização.



Batch: quantas imagens antes de cada atualização?



batch atual



Predizer → calcular loss → atualizar pesos

Batch maior

Pode aproveitar melhor a GPU, mas exige mais memória.

Batch menor

Consome menos memória, porém produz atualizações mais frequentes e ruidosas.

Se ocorrer falta de memória, reduzir o batch é uma das primeiras tentativas.



320



640



960

Maior resolução

Pode preservar mais detalhes de objetos pequenos, com maior consumo de memória e tempo.

Menor resolução

Reduz custo, mas detalhes importantes podem desaparecer durante o redimensionamento.

Compare resoluções: maior não significa automaticamente melhor.

device: onde o treinamento será executado?



Automático
device=None

CPU
device="cpu"

GPU CUDA
device=0

Apple Silicon
device="mps"

GPU disponível

O treinamento normalmente será muito mais rápido que na CPU.

Sempre confirme

Observe no log qual dispositivo foi selecionado e quanta memória está sendo usada.



```
results = model.train(  
    data="data.yaml",  
    epochs=50,  
    imgsz=640,  
    project="runs_aula09",  
    name="nano_640_exp01"  
)
```

project

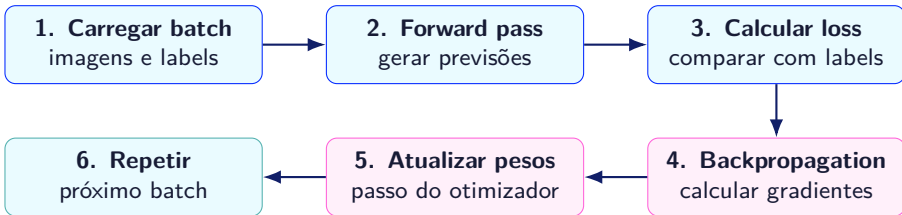
Pasta que reúne uma família de execuções.

name

Nome descritivo da configuração executada.

Um bom nome registra modelo, resolução e número do experimento.

O que acontece em uma iteração de treinamento?



Uma época termina quando todos os batches previstos foram processados.



box_loss

Relacionada ao ajuste da localização e das dimensões das caixas.

cls_loss

Relacionada à identificação das classes dos objetos.

dfl_loss

Componente associada à regressão das caixas no detector.

Cuidado ao interpretar

Uma loss menor costuma indicar melhora dentro do mesmo componente e experimento. Não compare diretamente componentes com escalas e funções diferentes.



Train

- ▶ gera previsões;
- ▶ calcula perdas;
- ▶ calcula gradientes;
- ▶ atualiza os pesos.

Val

- ▶ gera previsões;
- ▶ compara com labels;
- ▶ calcula métricas;
- ▶ não ensina com essas imagens.

A validação pergunta: **o aprendizado obtido no treino funciona em exemplos separados?**



best.pt

Checkpoint selecionado pelo critério de desempenho acompanhado durante o treinamento.

Uso comum: avaliação e inferência do melhor modelo observado.

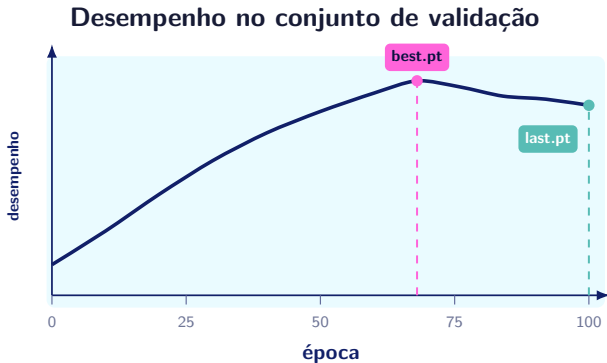
last.pt

Estado salvo ao final da execução, correspondente à última época processada.

Uso comum: retomar um treinamento interrompido.

A última época não é necessariamente a de melhor generalização.

Onde aparecem best.pt e last.pt?



best.pt

Corresponde ao **melhor desempenho de validação** observado durante o treinamento.

last.pt

Corresponde ao estado salvo na **última época** processada.

No exemplo, o melhor ponto ocorreu antes do final.

Os dois arquivos podem coincidir, mas isso não é obrigatório.

Onde ficam os resultados do treinamento?



```
runs_aula09/  
'-- nano_640_exp01/  
  |-- weights/  
  |   |-- best.pt  
  |   '-- last.pt  
  |-- results.csv  
  |-- results.png  
  |-- confusion_matrix.png  
  '-- val_batch*_pred.jpg
```

Pesos

Checkpoints para inferência, validação ou retomada.

Evidências

Métricas, curvas, exemplos de labels e previsões.

Use sempre o caminho informado no log; a organização pode mudar conforme a configuração.



```
from ultralytics import YOLO

model = YOLO(
    "runs_aula09/nano_640_exp01/"
    "weights/best.pt"
)

metrics = model.val(
    data="data.yaml",
    plots=True
)

print(metrics.box.map)
print(metrics.box.map50)
```

`box.map`

mAP calculada na faixa de IoUs de 0,50 a 0,95.

`box.map50`

mAP calculada com IoU igual a 0,50.

Validação e predição respondem a perguntas diferentes



	Validação	Predição
Pergunta	Quanto o modelo acerta em dados rotulados?	Como o modelo se comporta em novas imagens?
Labels	Necessários para comparar com o ground truth.	Não são necessários para gerar detecções.
Saída	Precision, recall, mAP, matriz e curvas.	Caixas, classes, confidências e imagens anotadas.
Uso	Avaliação quantitativa controlada.	Inspeção qualitativa em condições reais.

Um projeto confiável combina métricas com inspeção visual de exemplos novos.



Precision

Das detecções realizadas, quantas estavam corretas?

$$P = \frac{TP}{TP + FP}$$

Baixa precision: muitos falsos positivos.

Recall

Dos objetos existentes, quantos foram encontrados?

$$R = \frac{TP}{TP + FN}$$

Baixo recall: muitos objetos perdidos.

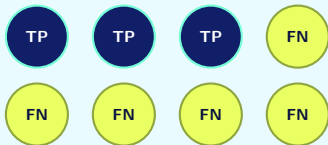
O limiar de confiança pode alterar o equilíbrio entre falsos positivos e falsos negativos.

Exemplo 1: alta precision, baixo recall



Existem 8 capacetes na imagem

O modelo encontra somente os casos mais evidentes.



Falsos alarmes:

FP = 0

3 encontrados 5 perdidos nenhum alarme falso

1. Conte os resultados

$$TP = 3, \quad FP = 0, \quad FN = 5$$

2. Precision alta

$$P = \frac{TP}{TP + FP} = \frac{3}{3 + 0} = 100\%$$

Leitura: o modelo realizou 3 detecções e todas estavam corretas.

3. Recall baixo

$$R = \frac{TP}{TP + FN} = \frac{3}{3 + 5} = 37,5\%$$

Leitura: dos 8 capacetes existentes, somente 3 foram encontrados.

Exemplo 2: alto recall, baixa precision



Existem 8 capacetes na imagem

O modelo aceita muitos candidatos como detecção.



Alarmes sem objeto real



7 encontrados 1 perdido 5 alarmes falsos

1. Conte os resultados

$$TP = 7, \quad FP = 5, \quad FN = 1$$

2. Precision baixa

$$P = \frac{TP}{TP + FP} = \frac{7}{7 + 5} = 58,3\%$$

Leitura: das 12 detecções realizadas, apenas 7 estavam corretas.

3. Recall alto

$$R = \frac{TP}{TP + FN} = \frac{7}{7 + 1} = 87,5\%$$

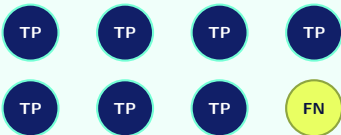
Leitura: dos 8 capacetes existentes, 7 foram encontrados.

Exemplo 3: precision e recall altos



Existem 8 capacetes na imagem

O modelo encontra quase todos sem exagerar nas detecções.



Alarme sem objeto real



7 encontrados 1 perdido 1 alarme falso

1. Conte os resultados

$$TP = 7, \quad FP = 1, \quad FN = 1$$

2. Precision alta

$$P = \frac{TP}{TP + FP} = \frac{7}{7 + 1} = 87,5\%$$

Leitura: das 8 detecções realizadas, 7 estavam corretas.

3. Recall alto

$$R = \frac{TP}{TP + FN} = \frac{7}{7 + 1} = 87,5\%$$

Leitura: dos 8 capacetes existentes, 7 foram encontrados.



1. Valores necessários

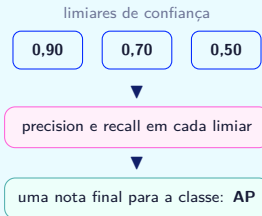
No conjunto de validação, precisamos de:

- ▶ classes e caixas reais;
- ▶ classes e caixas preditas;
- ▶ confiança de cada predição;
- ▶ um limiar de **IoU** para decidir se a caixa está correta.

Esses valores permitem identificar **TP, FP e FN**.

2. AP de cada classe

O sistema testa diferentes limiares de confiança. Em cada um, calcula **precision** e **recall** para a classe.



Exemplo: $AP_{\text{capacete}} = 82\%$.

3. Exemplo de mAP50

O número **50** significa que uma predição precisa ter $IoU \geq 0,50$ com a caixa real.

$$IoU = 0,62 \geq 0,50 \Rightarrow \text{TP}$$

capacete
82%

boné
74%

pessoa
88%

A **mAP50** é a média das APs:

$$mAP50 = \frac{82 + 74 + 88}{3} = 81,3\%$$

O 50 representa a **IoU mínima**, e não 50% de acerto.

Matriz de confusão: onde o modelo está errando?



		Classe predita →		
		capacete	boné	fundo
Classe real	capacete	42	8	5
	boné	11	31	7
	fundo	4	6	—

● correto ● confusão ● fundo

Diagonal principal

Os valores **42** e **31** representam acertos nas classes capacete e boné.

Maior confusão

11 bonés foram preditos como capacetes, enquanto **8 capacetes** foram preditos como bonés.

Ação sugerida

Revisar as anotações e adicionar exemplos variados que evidenciem as diferenças visuais entre as duas classes.

Leia cada linha como a classe real e cada coluna como a predição do modelo.

Analizando a matriz: classe capacete



Positivo: capacete
Negativo: boné ou fundo

		Classe predita →		
		capacete	boné	fundo
Classe real	capacete	42 TP	8 FN	5 FN
	boné	11 FP	31 TN	7 TN
	fundo	4 FP	6 TN	— N/A

Contagens: capacete vs. demais classes

TP 42
FP 11 + 4 = 15
FN 8 + 5 = 13
TN 31 + 7 + 6 = 44*

Precision

$$\frac{TP}{TP + FP} = \frac{42}{42 + 15} = \frac{42}{57} \approx 73,7\%$$

De 57 predições de capacete, 42 estavam corretas.

Recall

$$\frac{TP}{TP + FN} = \frac{42}{42 + 13} = \frac{42}{55} \approx 76,4\%$$

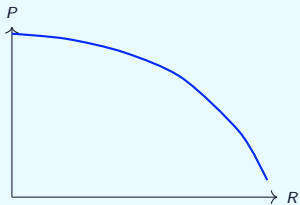
Dos 55 capacetes reais, o modelo encontrou 42.

*TN é apenas ilustrativo: em detecção, regiões de fundo não são enumeradas e essa medida não costuma ser reportada.

Curvas: observe relações, não apenas um número final

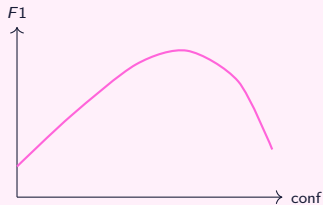


Precision-Recall



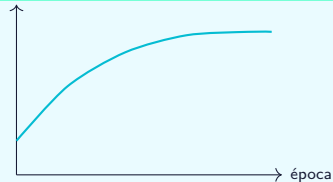
Mostra o compromisso entre encontrar mais objetos e evitar falsos positivos.

F1-Confidence



Ajuda a investigar o equilíbrio entre precision e recall em cada limiar.

Resultados por época



Permite observar evolução, estabilização e possível divergência.

Abra os artefatos: eles fazem parte da avaliação



`results.png`
losses e métricas por época

`confusion_matrix.png`
confusões entre classes e fundo

`BoxPR_curve.png`
relação precision-recall

`val_batch*_labels.jpg`
ground truth carregado

`val_batch*_pred.jpg`
previsões do modelo

Compare labels e previsões: caixas ausentes ou erradas podem denunciar o dataset.



Underfitting

Sinal: desempenho fraco até no conjunto de treino.

- ▶ poucas épocas;
- ▶ modelo ou configuração insuficiente;
- ▶ padrões difíceis de aprender.

Overfitting

Sinal: treino melhora enquanto a validação estagna ou piora.

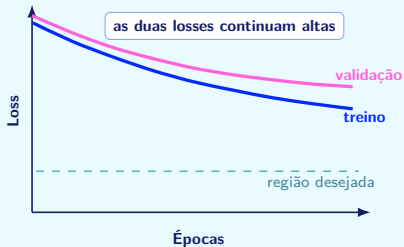
- ▶ poucos dados diversos;
- ▶ repetição e vazamento;
- ▶ treinamento além do útil.

O objetivo não é memorizar o treino; é funcionar em exemplos que o modelo nunca viu.

Como as curvas revelam underfitting e overfitting

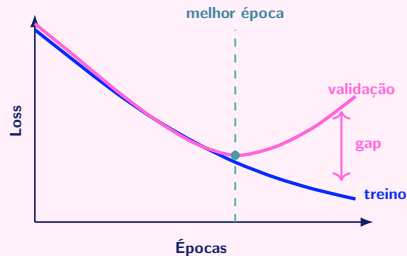


Underfitting: o modelo ainda não aprendeu



Leitura: desempenho fraco tanto no treino quanto na validação.

Overfitting: o modelo começa a memorizar



Leitura: o treino melhora, mas a validação volta a piorar.

— loss de treino — loss de validação

Quando o problema não é o modelo



labels incorretos

poucas imagens

classes des-
balanceadas

train e val
parecidos

domínio diferente

imagens de
baixa qualidade

classe inconsistente

Ordem de investigação

Visualize labels, conte classes, procure duplicatas e analise erros por cenário antes de alterar muitos hiperparâmetros.



Outra câmera
sensor e compressão

Outra iluminação
sombra, noite e reflexo

Outro fundo
cenário mais complexo

Escala
objetos pequenos e distantes

Oclusão
objetos parcialmente visíveis

Movimento
desfoque e novos ângulos

Não escolha apenas exemplos fáceis

Registre sucessos e falhas. Os erros mais frequentes indicam onde coletar dados ou revisar a definição do problema.



Comparação interpretável

Baseline:

nano, 640, 50 épocas, dataset v1

Experimento 2:

nano, 960, 50 épocas, dataset v1

Apenas a resolução mudou.

Comparação ambígua

Experimento 3:

modelo maior, 960, 100 épocas, dataset v2,
nova augmentation

Se o resultado mudar, não saberemos qual
decisão causou o efeito.

Registre configuração, resultado, tempo e observações de cada execução.



versões das bibliotecas

versão do dataset

modelo e check-
point inicial

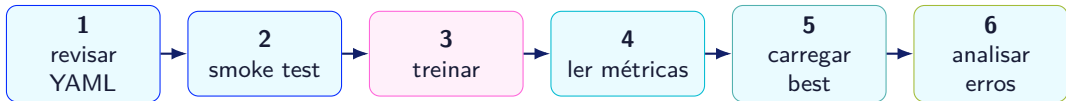
configuração e seed

hardware e duração

métricas e ar-
tefatos finais

Documentação científica

“Funcionou uma vez” não informa como repetir, comparar ou explicar o resultado.



Meta da prática

Produzir uma execução completa e documentada. A meta não é obter um modelo perfeito com um dataset didático pequeno.



```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")

model.train(
    data="data.yaml",
    epochs=3,
    imgsz=640,
    project="runs_aula09",
    name="smoke_test"
)
```

Confirme

- ▶ imagens localizadas;
- ▶ labels carregados;
- ▶ classes corretas;
- ▶ primeira época concluída.

Não avalie qualidade ainda

Três épocas servem para testar o fluxo, não para concluir se o detector é bom.



```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")

results = model.train(
    data="data.yaml",
    epochs=20,
    imgsz=640,
    project="runs_aula09",
    name="grupo_01_exp01"
)
```

Durante a execução

- ▶ não feche o processo;
- ▶ acompanhe o tempo;
- ▶ observe mensagens;
- ▶ registre anomalias.

Adapte o nome do grupo e o número do experimento.

Prática 3: acompanhe o que o log está informando



Preparação

- ▶ modelo carregado;
- ▶ dataset encontrado;
- ▶ quantidade de classes;
- ▶ labels e cache;
- ▶ device escolhido.

Treinamento

- ▶ época atual;
- ▶ memória da GPU;
- ▶ componentes da loss;
- ▶ instâncias no batch;
- ▶ tempo restante.

Validação

- ▶ precision;
- ▶ recall;
- ▶ mAP50;
- ▶ mAP50–95;
- ▶ melhor época.

Se o log indicar zero labels ou caminhos ausentes, interrompa e corrija o dataset.



```
from ultralytics import YOLO

trained = YOLO(
    "runs_aula09/grupo_01_exp01/"
    "weights/best.pt"
)

results = trained.predict(
    source="imagens_novas",
    save=True,
    conf=0.25
)
```

Use dados realmente novos

Não escolha imagens copiadas dos subconjuntos de treino ou validação.

Guarde as evidências

Separe exemplos de sucesso e de erro para a próxima aula.

Prática 5: transforme falhas em hipóteses



Falso positivo
detectou onde
não havia objeto

Falso negativo
não encontrou um
objeto existente

Classe incorreta
localizou, mas con-
fundiu a categoria

Caixa imprecisa
posição ou dimen-
são inadequada

Cenário difícil
escala, oclusão,
luz ou domínio

Para cada erro, registre

imagem, classe esperada, previsão, condição visual e uma hipótese de melhoria nos dados ou no experimento.



Configuração

- ▶ quantidade de imagens: _____
- ▶ classes: _____
- ▶ modelo inicial: _____
- ▶ epochs: _____
- ▶ imgsz: _____

Resultado

- ▶ precision: _____
- ▶ recall: _____
- ▶ mAP50: _____
- ▶ mAP50-95: _____
- ▶ principal erro: _____

Próxima melhoria proposta: _____



- 1 O que é transfer learning?
- 2 O que são epochs?
- 3 Como o batch afeta o treinamento?
- 4 Por que usamos validação?
- 5 O que representa `best.pt`?
- 6 Se o resultado estiver ruim, o que investigar primeiro?



Treinar produz pesos; avaliar produz evidências. Um detector útil nasce do ciclo entre dados, experimento e análise de erros.

Leve para a Aula 10

- ▶ o checkpoint `best.pt`;
- ▶ cinco exemplos de sucesso e cinco exemplos de erro;
- ▶ precision, recall, mAP50 e mAP50–95;
- ▶ uma hipótese de melhoria para o projeto.

Próxima aula: aplicação, tracking, exportação e projeto final.